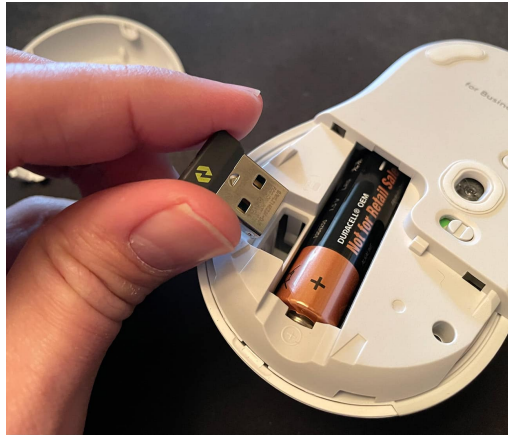


Custom Mouse Driver Project

Gesture Recognition with USB and Serial Mouse (Linux Kernel Module)



Authors:

Alp Eren Erdur (ID: S034494)

Murat Tuna Özen (ID: S041084)

Course: Operating Systems (CS350)

Instructor: Dr İsmail Arı

Department of Computer Science / Engineering

Özyeğin University

January 2, 2026

Contents

1	Introduction	2
2	Basic OS Concepts	2
2.1	Kernel space vs user space	2
2.2	Device file: /dev/mousepattern	2
2.2.1	Creating /dev/mousepattern in the driver	3
2.2.2	Writing an event to /dev/mousepattern	3
2.2.3	Reading the event in user space	3
2.3	Wait queue and mutex (why we need them)	4
2.4	Interrupts (IRQ) vs Polling	4
2.4.1	IRQ example from our serial mouse driver	4
3	System Overview (How everything connects)	5
4	Common Design: Event Buffer and Read Function	6
4.1	Ring buffer push (example from USB driver)	6
4.2	Read side (example from USB driver)	8
5	USB Mouse Driver (Input Subsystem)	8
5.1	Why Input Subsystem?	8
5.2	Connecting to a mouse	9
5.3	Main event logic: click counting + gesture	9
5.4	Creating the device file	10
6	Serial Mouse Driver (serio + packet decoding) — serial_mouse_pattern.c	10
6.1	Hardware setup	10
6.2	How serial mouse data arrives (3-byte packets)	11
6.2.1	Synchronization problem and fix	11
6.3	Decoding X/Y movement from the packet	11
6.4	Where the serial driver runs (interrupt callback)	12
7	User-space Listener — listener.c	12
7.1	Reading messages	12
7.2	Starting and stopping music	13
8	Build and Run	13
8.1	Requirements	13
8.2	Building the kernel modules	13
8.3	Loading / unloading	13
8.4	Running the listener	13
9	Results	14
10	Limitations and Small Improvements	14
11	Conclusion	14
A	Appendix: File List (What each file is for)	15

1 Introduction

In this project we wrote a **custom mouse driver** in Linux. The driver runs inside the **kernel** and listens to mouse input. Our goal was simple: detect a mouse “gesture” and then send a message to a user program. That user program starts or stops music.

We implemented the same idea in two ways:

- **USB mouse driver:** uses the Linux **Input Subsystem**.
- **Serial mouse driver:** uses the Linux **serio** layer and decodes the old 3-byte mouse packets.

The demo is fun but the point is OS-level: kernel vs user space, device files, interrupts/events, and safe communication.

Project idea in one sentence

Right click + move *Right* → *Left* → *Up* → *Down* = **START_MUSIC**.
Triple right click = **STOP_MUSIC**.

2 Basic OS Concepts

2.1 Kernel space vs user space

Linux has two worlds:

- **User space:** normal programs (terminal, browser). They cannot touch hardware directly.
- **Kernel space:** the OS core. Drivers run here and can access devices safely (and dangerously if you mess up).

Our mouse drivers run in **kernel space**. The music player runs in user space. So we needed a bridge between them.

2.2 Device file: /dev/mousepattern

We used `/dev/mousepattern` as a communication channel between **kernel space** and **user space**.

Our mouse driver runs inside the kernel and detects gestures. However, a kernel driver should **not** start or stop applications such as a music player. That kind of work belongs to user space.

For this reason, instead of controlling music directly, the driver writes simple text messages (for example `START_MUSIC` or `STOP_MUSIC`) into a character device file called `/dev/mousepattern`. The user-space program reads these messages and decides what to do.

In our project, `/dev/mousepattern` behaves like a normal file: the user program only uses `open()` and `read()`.

2.2.1 Creating /dev/mousepattern in the driver

The device file is created during module initialization in `usb_mouse_pattern.c`. When the kernel executes `device_create()`, the file `/dev/mousepattern` appears automatically.

Listing 1: Creating `/dev/mousepattern` (`usb_mouse_pattern.c`)

```
1 #define DEV_NAME "mousepattern" // Name of the character device (/dev/
   mousepattern)
2
3 static int __init my_init(void) {
4
5     // Allocate a unique major/minor number for our character device
6     alloc_chrdev_region(&gdev->devt, 0, 1, DEV_NAME);
7
8     // Initialize and register the character device with our file
   operations
9     cdev_init(&gdev->cdev, &my_fops);
10    cdev_add(&gdev->cdev, gdev->devt, 1);
11
12    // Create a device class under /sys/class/
13    gdev->class = class_create(DEV_NAME);
14
15    // Create the actual device file: /dev/mousepattern
16    device_create(gdev->class, NULL, gdev->devt, NULL, DEV_NAME);
17
18    // Register our input handler to start receiving mouse events
19    input_register_handler(&pattern_handler);
20
21    return 0; // Module loaded successfully
22 }
```

Simple explanation:

- The driver asks the kernel for a device number.
- It registers its `read()` function.
- Linux automatically creates the file `/dev/mousepattern`.

2.2.2 Writing an event to /dev/mousepattern

When a gesture is detected, the driver does not control music directly. Instead, it writes a short message into an internal buffer.

Listing 2: Writing an event message (`usb_mouse_pattern.c`)

```
1 event_buf_push("START_MUSIC\n");
```

This message later becomes available to user space through `/dev/mousepattern`.

2.2.3 Reading the event in user space

When the user program calls `read()`, the kernel executes the driver's `read` function. If there is no data, the process sleeps. When data exists, characters are copied safely to user space.

Listing 3: Read function for /dev/mousepattern (usb_mouse_pattern.c)

```
1 static ssize_t my_read(struct file *file, char __user *buf,
2                       size_t count, loff_t *ppos) {
3     wait_event_interruptible(gdev->read_queue,
4                             gdev->head != gdev->tail);
5     // data is copied to user space here
6     return copied;
7 }
```

Result: the user program simply reads text from /dev/mousepattern and reacts accordingly (for example, starting or stopping music).

2.3 Wait queue and mutex (why we need them)

Two small but important kernel tools:

- **Wait queue:** If there is no data to read, the user program should sleep (not waste CPU). The driver wakes it up when new data arrives.
- **Mutex:** Driver code can run from different contexts (event/interrupt + read). Mutex prevents mixing the buffer pointers.

2.4 Interrupts (IRQ) vs Polling

There are two common ways to read data from hardware: **polling** and **interrupts (IRQ)**.

Polling means the CPU constantly checks the device: “Did something happen? Did something happen?” This wastes CPU time, because most of the time nothing changes.

Interrupts (IRQ) work differently. The CPU does its normal work, and the hardware sends an interrupt signal only when new data arrives. This is more efficient and is the standard approach in device drivers.

In our project, both the USB mouse driver and the serial mouse driver use the interrupt-based model provided by the Linux kernel. We do not poll the mouse manually.

2.4.1 IRQ example from our serial mouse driver

The function below is called automatically by the kernel whenever a new byte arrives from the serial mouse. This function runs in interrupt context.

Listing 4: IRQ-based input handling (serial_mouse_pattern.c)

```
1 // INTERRUPT HANDLER: runs when new serial mouse byte arrives
2 static irqreturn_t mousepattern_serio_interrupt(struct serio *serio,
3                                                 unsigned char data,
4                                                 unsigned int flags)
5 {
6     struct mousepattern_dev *dev = serio_get_drvdata(serio);
7
8     if (!dev) return IRQ_HANDLED;
9
10    // each interrupt gives 1 byte, store it into packet buffer
11    dev->packet[dev->packet_idx++] = data;
12
13    // first byte must have bit 6 = 1 (0x40), otherwise we lost sync
```

```

14     if (dev->packet_idx == 1 && !(data & 0x40)) {
15         dev->packet_idx = 0;    // reset, ignore this byte
16         return IRQ_HANDLED;
17     }
18
19     // when we collected 3 bytes, packet is complete
20     if (dev->packet_idx == 3) {
21         // decode + click/gesture logic happens here (same function)
22
23         dev->packet_idx = 0;    // ready for next packet
24     }
25
26     return IRQ_HANDLED;
27 }

```

Simple explanation:

- The driver does not ask the mouse for data. The mouse interrupts the CPU when data is ready.
- Each interrupt brings one byte from the serial mouse.
- The driver rebuilds packets step by step inside the IRQ handler.
- This approach saves CPU time compared to polling.

3 System Overview (How everything connects)

The flow is always the same:

1. Mouse generates input.
2. Kernel driver receives input (USB input events or serial packets).
3. Driver updates a small state machine and decides if a gesture is completed.
4. Driver pushes a text message into `/dev/mousepattern`.
5. User-space program (`listener.c`) reads that text and runs commands (`mpg123`, `killall`).

```
ubuntu@ubuntu: ~/fare_projesi_USB
File Edit View Search Terminal Help

[KLİK] Tık 1...
[KLİK] Tık 2...
[KLİK] Tık 3! (KOMUT ALINDI)

MÜZİK DURDURULUYOR...

mpg123: death by SIGTERM

[0:02] Decoding of Bu Bayrak - Turkish Patriotic Song (This Flag).mp3 finished.
^C
ubuntu@ubuntu:~/fare_projesi_USB$ ^C
ubuntu@ubuntu:~/fare_projesi_USB$ sudo ./mouse_listener
*****
HAREKET ALGILAMA MODU AKTIF
*****
1. BASLATMAK ICIN: Sag Tık Basili Tut + (Sag-Sol-Yukari-Asagi)
2. DURDURMAK ICIN: Sag Tika 3 kere hizlica bas

[KLİK] Tık 1...
[KLİK] Tık 1...
[HAREKET] --> Sağa Gidildi (1/4 Tamam)
[HAREKET] <-- Solu Gidildi (2/4 Tamam)
[HAREKET] ^ Yukarı Gidildi (3/4 Tamam)

MÜZİK DURDURULUYOR...

ŞİFRE DOĞRU! MÜZİK BAŞLIYOR<<<
High Performance MPEG 1.0/2.0/2.5 Audio Player for Layers 1, 2 and 3
    version 1.32.5; written and copyright by Michael Hipp and others
    free software (LGPL) without any warranty but with best wishes

Directory: /home/ubuntu/fare_projesi_serial/

Terminal control enabled, press 'h' for listing of keys and functions.

Playing MPEG stream 1 of 1: Bu Bayrak - Turkish Patriotic Song (This Flag).mp3 ...

MPEG 1.0 L III vbr 44100 j-s
```

Figure 1: Demo output: the gesture steps are detected and music starts/stops.

4 Common Design: Event Buffer and Read Function

Both drivers store outgoing messages in a small text buffer and expose them through `/dev/mousepattern`.

4.1 Ring buffer push (example from USB driver)

The function below writes a string into the circular buffer and wakes up any reader. It writes character-by-character so we can keep it very simple.

Listing 5: Event push function (`usb_mouse_pattern.c`)

```
1 static void event_buf_push(const char *msg) {
2     size_t len = strlen(msg);    // length of the incoming message string
3     size_t i;                    // loop counter
4 }
```

```

5     mutex_lock(&gdev->lock);    // lock the buffer to prevent race
        conditions
6
7     for(i = 0; i < len; i++) {    // loop over each character in the
        message
8
9         // write one character into the buffer at position 'head'
10        gdev->event_buf[gdev->head] = msg[i];
11
12        // move head forward by one position
13        // '%' makes the buffer wrap around (circular buffer)
14        gdev->head = (gdev->head + 1) % EVENT_BUF_SIZE;
15
16        // if head reaches tail, buffer is full
17        // move tail forward and drop the oldest character
18        if(gdev->head == gdev->tail)
19            gdev->tail = (gdev->tail + 1) % EVENT_BUF_SIZE;
20    }
21
22    mutex_unlock(&gdev->lock);    // unlock buffer after
        writing
23    wake_up_interruptible(&gdev->read_queue); // wake up reader if it is
        sleeping
24 }
25
26 }

```

Simple explanation: This function stores an event message into a circular buffer, one character at a time. The `head` pointer shows where to write next, and `tail` shows where to read. If the buffer becomes full, the oldest data is overwritten. After writing, the reader process is woken up so it can immediately read the new event.

Explanation using the actual project code:

Listing 6: Ring buffer write logic from our project

```

1  for(i = 0; i < len; i++) {
2      gdev->event_buf[gdev->head] = msg[i];
3      gdev->head = (gdev->head + 1) % EVENT_BUF_SIZE;
4
5      if(gdev->head == gdev->tail)
6          gdev->tail = (gdev->tail + 1) % EVENT_BUF_SIZE;
7  }

```

- The loop writes the event message (`msg`) into the buffer one character at a time. `gdev->event_buf[gdev->head]` is the exact place where the next character is stored.
- After writing a character, `head` is increased. The modulo operation makes the buffer wrap around, so it behaves like a circle.
- If `head` becomes equal to `tail`, it means the buffer is full. In this case, we move `tail` forward and discard the oldest data.
- This logic guarantees that new events are always stored, even if old events are lost.

4.2 Read side (example from USB driver)

The user program does `read(fd, ...)`. If no data exists, we put it to sleep. When data arrives, we copy bytes to user space using `put_user`.

Listing 7: Read function (`usb_mouse_pattern.c`)

```
1 static ssize_t my_read(struct file *file, char __user *buf,
2                       size_t count, loff_t *ppos) {
3     size_t copied = 0;    // how many char we copied to user
4     char c;              // one character from buffer
5
6     // if buffer empty, process sleep and wait new event
7     if(wait_event_interruptible(gdev->read_queue,
8                                (gdev->head != gdev->tail)))
9         return -ERESTARTSYS; // signal interrupted
10
11     mutex_lock(&gdev->lock); // lock buffer for safe read
12
13     // read until user buffer full or ring buffer empty
14     while(copied < count && gdev->head != gdev->tail) {
15
16         c = gdev->event_buf[gdev->tail]; // take one char from tail
17         gdev->tail = (gdev->tail + 1) % EVENT_BUF_SIZE; // move tail
18
19         mutex_unlock(&gdev->lock); // unlock before copy to user
20
21         // copy one char from kernel to user space
22         if(put_user(c, buf + copied))
23             return -EFAULT; // error if copy fail
24
25         mutex_lock(&gdev->lock); // lock again for next char
26         copied++;                // one more char copied
27     }
28
29     mutex_unlock(&gdev->lock); // unlock buffer
30     return copied;            // return how many byte read
31 }
```

Why `put_user`? Kernel memory and user memory are different. Direct assignment like `buf[i] = c` is not allowed. So the kernel provides safe copy helpers.

5 USB Mouse Driver (Input Subsystem)

— `usb_mouse_pattern.c`

5.1 Why Input Subsystem?

USB is complicated at the low level. The good news: Linux already has a full USB stack. So we used the **Input Subsystem**, which delivers clean events like:

- `EV_KEY`: a button press/release
- `EV_REL`: relative movement (X/Y)

Our driver registers an `input_handler`. After that, the kernel calls our event function automatically.

5.2 Connecting to a mouse

We match devices using an ID table. We require:

- supports EV_KEY and EV_REL
- has BTN_LEFT
- has REL_X and REL_Y

This prevents connecting to random input devices.

5.3 Main event logic: click counting + gesture

The core is the `mouse_event` callback. We handle two things:

1. Right button presses (count 1,2,3 fast clicks)
2. Mouse movement while right button is held (gesture steps)

Listing 8: USB event handling (shortened)

```
1 static void mouse_event(struct input_handle *handle,
2                          unsigned int type, unsigned int code, int value)
3 {
4     // check mouse button events
5     if (type == EV_KEY && code == BTN_RIGHT) {
6
7         if (value == 1) { // right button pressed
8             gdev->right_button_pressed = true;
9
10            // check if clicks are close in time (fast click)
11            if (time_before(jiffies,
12                            gdev->last_click_time + msecs_to_jiffies(400)))
13                gdev->right_click_count++; // same click sequence
14            else
15                gdev->right_click_count = 1; // new click sequence
16
17            gdev->last_click_time = jiffies; // save click time
18
19            // if right button clicked 3 times fast
20            if (gdev->right_click_count == 3) {
21                event_buf_push("STOP_MUSIC\n"); // send stop command
22                gdev->right_click_count = 0; // reset counter
23            }
24
25            } else if (value == 0) { // right button released
26                gdev->right_button_pressed = false;
27
28                // if gesture was half done, reset it
29                if (gdev->combo_state != 0)
30                    event_buf_push("RESET\n");
31
32                gdev->combo_state = 0; // go back to start state
33            }
34    }
35 }
```

```

36 // check mouse movement events
37 if (type == EV_REL && gdev->right_button_pressed) {
38
39     // code 0 = X axis, code 1 = Y axis
40     // THRESHOLD used to ignore small jitter
41
42     if (gdev->combo_state == 0 && code == 0 && value > THRESHOLD) {
43         gdev->combo_state = 1;           // step 1 done
44         event_buf_push("ADIM_SAG\n");    // moved right
45
46     } else if (gdev->combo_state == 1 && code == 0 && value < -
47         THRESHOLD) {
48         gdev->combo_state = 2;           // step 2 done
49         event_buf_push("ADIM_SOL\n");    // moved left
50
51     } else if (gdev->combo_state == 2 && code == 1 && value < -
52         THRESHOLD) {
53         gdev->combo_state = 3;           // step 3 done
54         event_buf_push("ADIM_YUKARI\n"); // moved up
55
56     } else if (gdev->combo_state == 3 && code == 1 && value >
57         THRESHOLD) {
58         gdev->combo_state = 0;           // gesture completed
59         event_buf_push("START_MUSIC\n"); // start music
60     }
61 }
62 }

```

Small details we added:

- THRESHOLD is used to ignore tiny movements (mouse jitter).
- combo_state is a simple state machine: 0 (start) → 1 (right) → 2 (left) → 3 (up) → start music (down).

5.4 Creating the device file

In `my_init` we allocate a char device number, add `cdev`, create a class, and then: `device_create(..., "mousepattern")` creates `/dev/mousepattern`. After that, user programs can read events.

6 Serial Mouse Driver (serio + packet decoding) — serial_mouse_pattern.c

6.1 Hardware setup

Serial mice are old, and modern laptops do not have RS-232 ports. So we used:

- a serial mouse
- a USB-to-Serial adapter (FTDI style)



Serial mouse



Bu ürün: DigiTus Usb Seri Adaptör
- Rs232 Dönüştürücü - Usb 2.0
Tip A - Dsub 9M - Ftdi Ft232Rl...

USB-to-Serial adapter

Figure 2: Hardware used for the serial mouse test.

6.2 How serial mouse data arrives (3-byte packets)

A Microsoft serial mouse sends packets in 3 bytes. But bytes come as a stream, so we must group them correctly.

6.2.1 Synchronization problem and fix

If we start reading in the middle of a packet, everything becomes wrong. We used a simple rule: the first byte has bit 6 set (0x40).

Listing 9: Packet sync check (serial_mouse_pattern.c / mousepattern_serio_interrupt)

```

1 // store incoming byte into packet buffer
2 // packet_idx shows which byte we are filling now
3 dev->packet[dev->packet_idx++] = data;
4
5 // first byte of serial mouse packet must have bit 6 = 1 (0x40)
6 // if first byte is wrong, we lost sync
7 if (dev->packet_idx == 1 && !(data & 0x40)) {
8
9     // wrong start byte, reset packet index
10    dev->packet_idx = 0;
11
12    // ignore this byte and wait next interrupt
13    return IRQ_HANDLED;
14 }
```

Explanation: `data & 0x40` checks if bit 6 is 1. If not, we throw the byte away and wait for a real packet start.

6.3 Decoding X/Y movement from the packet

This part looks scary at first, but it is just bit masks and shifts. The movement bits are split across the bytes, so we combine them.

Listing 10: Movement decode (serial_mouse_pattern.c)

```
1 move_x = (signed char) (((dev->packet[0] & 0x03) << 6)
2 | (dev->packet[1] & 0x3F));
3 move_y = (signed char) (((dev->packet[0] & 0x0C) << 4)
4 | (dev->packet[2] & 0x3F));
```

- & 0x03 means “take only the last 2 bits”.
- << 6 shifts them to their real position.
- | (OR) merges the parts into one byte.
- We use `signed char` so movement can be negative (left/up).

After decoding, the gesture logic is basically the same as USB: threshold checks + `combo_state` steps. The serial driver also counts triple right click to stop music.

6.4 Where the serial driver runs (interrupt callback)

In the serial driver, our main function is called from: `mousepattern_serio_interrupt(...)`. This is the function that receives bytes as they arrive.

So the serial version feels more “low level” than USB, because we manually rebuild packets.

7 User-space Listener — `listener.c`

The user program is small and simple. It opens `/dev/mousepattern`, then it reads strings and reacts.

7.1 Reading messages

It uses `strstr` to check which message arrived. Example: if it sees `START_MUSIC`, it calls `play_music()`.

Listing 11: Listener main loop (shortened)

```
1 int fd = open("/dev/mousepattern", O_RDONLY);
2 while (1) {
3     int n = read(fd, buffer, sizeof(buffer)-1);
4     if (n <= 0) continue;
5     buffer[n] = '\0';
6
7     if (strstr(buffer, "START_MUSIC")) {
8         play_music();
9         sleep(2);
10    } else if (strstr(buffer, "STOP_MUSIC")) {
11        stop_music();
12        sleep(1);
13    }
14 }
```

7.2 Starting and stopping music

We used `mpg123` to play an MP3. To stop it, we do `killall mpg123`. We also run `mpg123` with `&` so the listener does not freeze.

Important setup note

In `listener.c`, the MP3 path is hard-coded. You must change it to your own file location before compiling.

8 Build and Run

8.1 Requirements

We used:

- build tools: `gcc`, `make`
- kernel headers: `linux-headers-$(uname -r)`
- audio player: `mpg123`

8.2 Building the kernel modules

We used a simple kernel module Makefile. Example:

- Serial build line: `obj-m += serial_mouse_pattern.o`
- USB build line: `obj-m += usb_mouse_pattern.o`

In our repository we switch that line depending on which module we want to compile. Then:

```
1 make
```

8.3 Loading / unloading

```
1 sudo insmod usb_mouse_pattern.ko
2 # or
3 sudo insmod serial_mouse_pattern.ko
4
5 dmesg | tail
6
7 sudo rmmod usb_mouse_pattern
8 # or
9 sudo rmmod serial_mouse_pattern
```

8.4 Running the listener

```
1 gcc listener.c -o mouse_listener
2 sudo ./mouse_listener
```

9 Results

We tested both versions with the same listener program. Figure 1 shows a successful run:

- The driver prints step messages (right, left, up, down).
- After the final step, it sends `START_MUSIC`.
- Triple right click sends `STOP_MUSIC`.

We also confirmed the device file exists: `/dev/mousepattern` was created after `insmod`.

10 Limitations and Small Improvements

We kept the project simple on purpose. Some limitations:

- Gesture uses fixed thresholds. Different mice may need different values.
- Listener uses `system()` which is not the cleanest method, but it is easy for a demo.
- The event buffer is small and we drop old events if it overflows (fine for our use).

Possible improvements (if we had more time):

- Make MP3 path configurable via command line.
- Add more gestures or a small configuration file.
- Better filtering (smoother gesture detection).

11 Conclusion

This project helped us understand how input reaches the Linux kernel and how drivers communicate with user programs. We wrote two drivers (USB and serial) that solve the same problem in different ways. In both cases, the design idea is the same: **detect a pattern in the kernel, send a message to user space, and trigger an action.**

A Appendix: File List (What each file is for)

- `usb_mouse_pattern.c`: USB mouse kernel module (Input Subsystem).
- `serial_mouse_pattern.c`: Serial mouse kernel module (serio + packet decode).
- `listener.c`: user-space program that reads `/dev/mousepattern` and controls music.
- `Makefile`: kernel module build file.
- `serialMouseVideo.mp4`: The test video of serial mouse.
- `usbMouseVideo.mp4`: The test video of USB mouse.